

SENAI SANTA CATARINA • PÓS-GRADUAÇÃO EM IA APLICADA

Classificador de Ambiente

Sistema embarcado inteligente com TinyML no Raspberry Pi Pico W

● Normal

● Presença

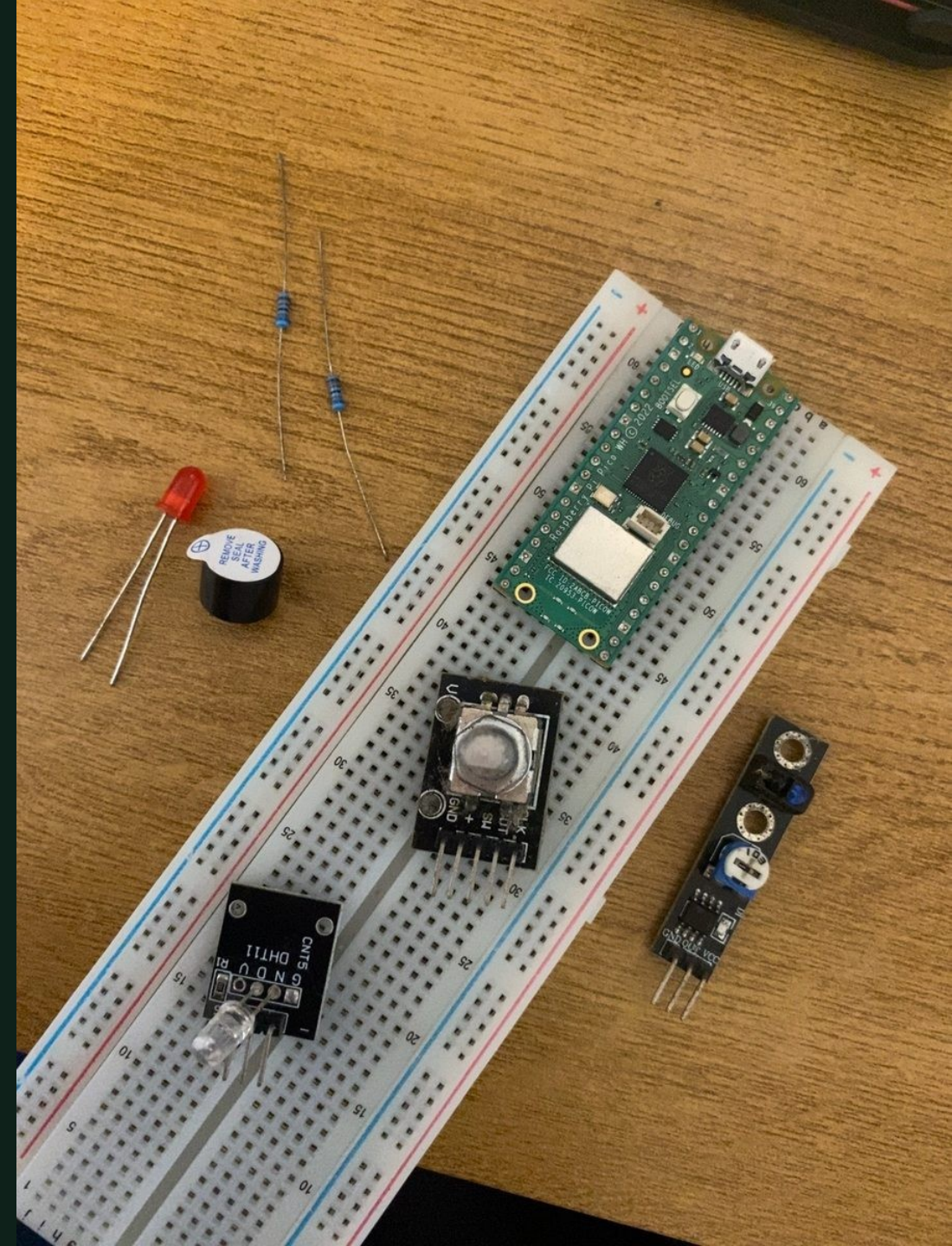
● Alerta

● Crítico

Integrantes Rafaela Santana • Miriam Aguiar • Sara Coutinho

Professor Rodrigo Kobashikawa Rosa

IA Embarcada e Modelos Compactos (489780) • Turma PG PGIA 2025/2 1



Por que monitorar o ambiente na borda?



Dependência da nuvem

Sem internet, a maioria dos sistemas para de alertar. Cada leitura percorre a rede antes de virar decisão.



Custo e infraestrutura

Gateways, servidores e assinaturas encarecem a instalação em salas, laboratórios e almoxarifados.



Regras cegas ao contexto

Um limite fixo ignora se há pessoas expostas: 32 °C com alguém presente é mais grave que em uma sala vazia.



PERGUNTA NORTEADORA

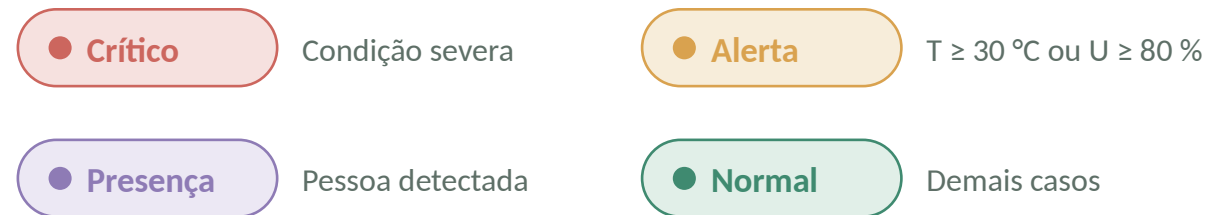
É possível embarcar um modelo de Machine Learning em um microcontrolador de baixo custo para classificar, em tempo real e de forma autônoma, o estado de um ambiente?

Do sensor à decisão, dentro da placa



Ciclo de leitura e inferência a cada 2 s • envio para a nuvem a cada 10 s (opcional)

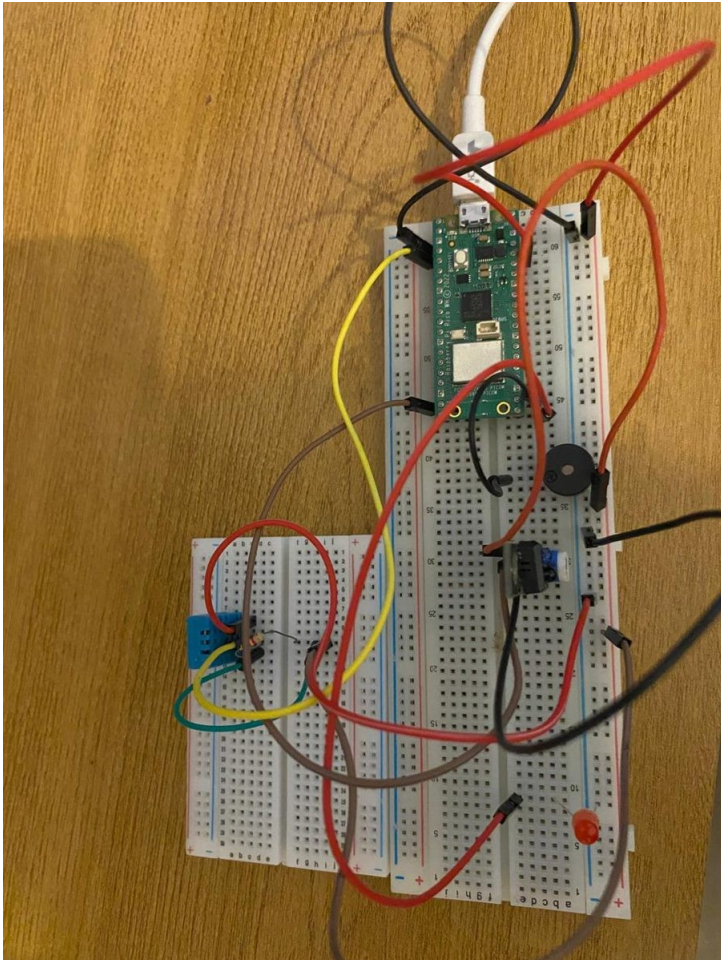
Quatro estados, por prioridade



Por que TinyML?

- Latência de microssegundos
- Funciona sem internet
- Menos tráfego e menor custo
- Dados processados no local

Montagem e pinagem final



DHT11 • DATA

Temperatura e umidade

GP2



Sensor IR • OUT

Presença (ativo em nível baixo)

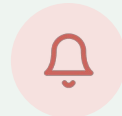
GP14



LED externo

Indicação visual

GP16



Buzzer passivo

Alarme sonoro de 2 kHz

GP17



3,3 V e GND comum entre todos os módulos. Um joystick testado no início saiu da montagem: menos fios, menos pontos de falha.

Cada sensor validado antes da IA

```
180         sleep(1000, 0.1)
181
182
183     print()
184
185     # O DHT11 não deve ser lido muito rapidamente
186     time.sleep(2)|
```

```
Shell >
Joystick X: 13187
Joystick Y: 64719
Botao joystick: solto
```

```
-----
Temperatura: 24 °C
Umidade: 70 %
Sensor IR: 1
Joystick X: 13875
Joystick Y: 64527
Botao joystick: solto
```

```
-----
Temperatura: 24 °C
Umidade: 70 %
Sensor IR: 1
Joystick X: 12242
Joystick Y: 64431
Botao joystick: solto
```



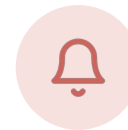
DHT11

Após ajustar o sinal para o GP2, passou a retornar temperatura e umidade de forma estável.



Sensor IR

Descoberto que é ativo em nível baixo: IR=0 indica presença e IR=1 indica ausência. A inversão foi levada ao firmware.



LED e buzzer

Acionados manualmente antes de integrar a lógica de classificação.



Hardware validado antes do treinamento

MicroPython no Thonny: leituras do DHT11, do IR e do joystick a cada 2 s

235 medições reais com o próprio protótipo

235

medições válidas

79

combinações únicas

23

medições com presença

156

repetições removidas

Entradas do modelo

temperatura_c 0 a 50 °C

umidade_pct 0 a 100 %

presenca 1 = detectada · 0 = ausente

Conversão do IR: IR=0 → presenca=1 · IR=1 → presenca=0

Rotulagem por prioridade (a primeira regra satisfeita vence)

● Crítico

$T \geq 35\text{ °C}$ ou $U \geq 90\%$, ou presença com $T \geq 32\text{ °C}$ ou $U \geq 85\%$

● Alerta

$T \geq 30\text{ °C}$ ou $U \geq 80\%$

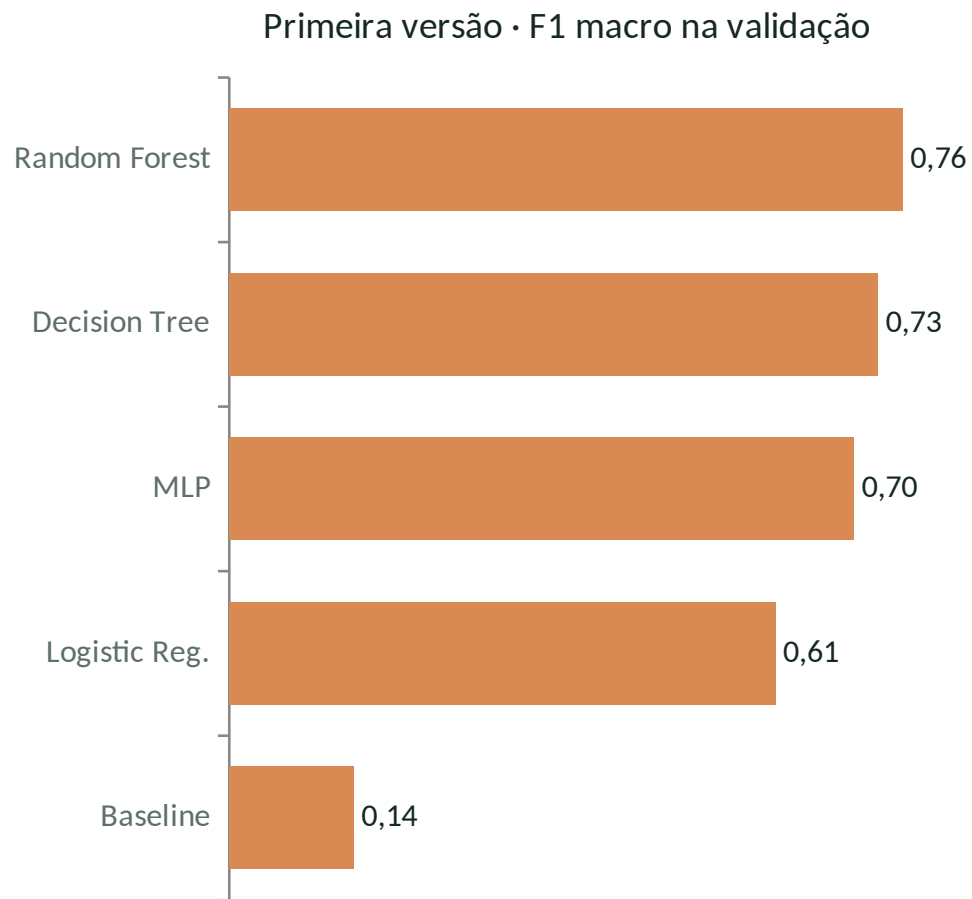
● Presença

Presença detectada, sem condição anterior

● Normal

Demais leituras válidas

A primeira versão revelou os problemas



F1 macro de apenas 0,73

Árvore com 17 nós; Random Forest e MLP também ficaram abaixo de 0,76.



Só 5 exemplos da classe presença

A validação teve de usar 2 folds; o teste tinha 20 amostras e apenas 1 de presença.



Nenhuma leitura com $T \geq 35^{\circ}\text{C}$

Uma árvore não aprende um limiar que nunca viu: divergia das regras em 23 % da grade de entradas.



100 % no teste não provava nada

Com 20 amostras únicas, o acerto total era estatisticamente frágil.

Do teste inicial à versão final

	VERSÃO INICIAL		VERSÃO FINAL
Registros no dataset	235	→	1.000
Combinações únicas	79	→	844
Exemplos por classe	5 a 135	→	250 cada
Validação cruzada	2 folds	→	5 folds
Amostras de teste	20	→	127
Árvore de decisão	17 nós	→	25 nós

F1 MACRO DA ÁRVORE

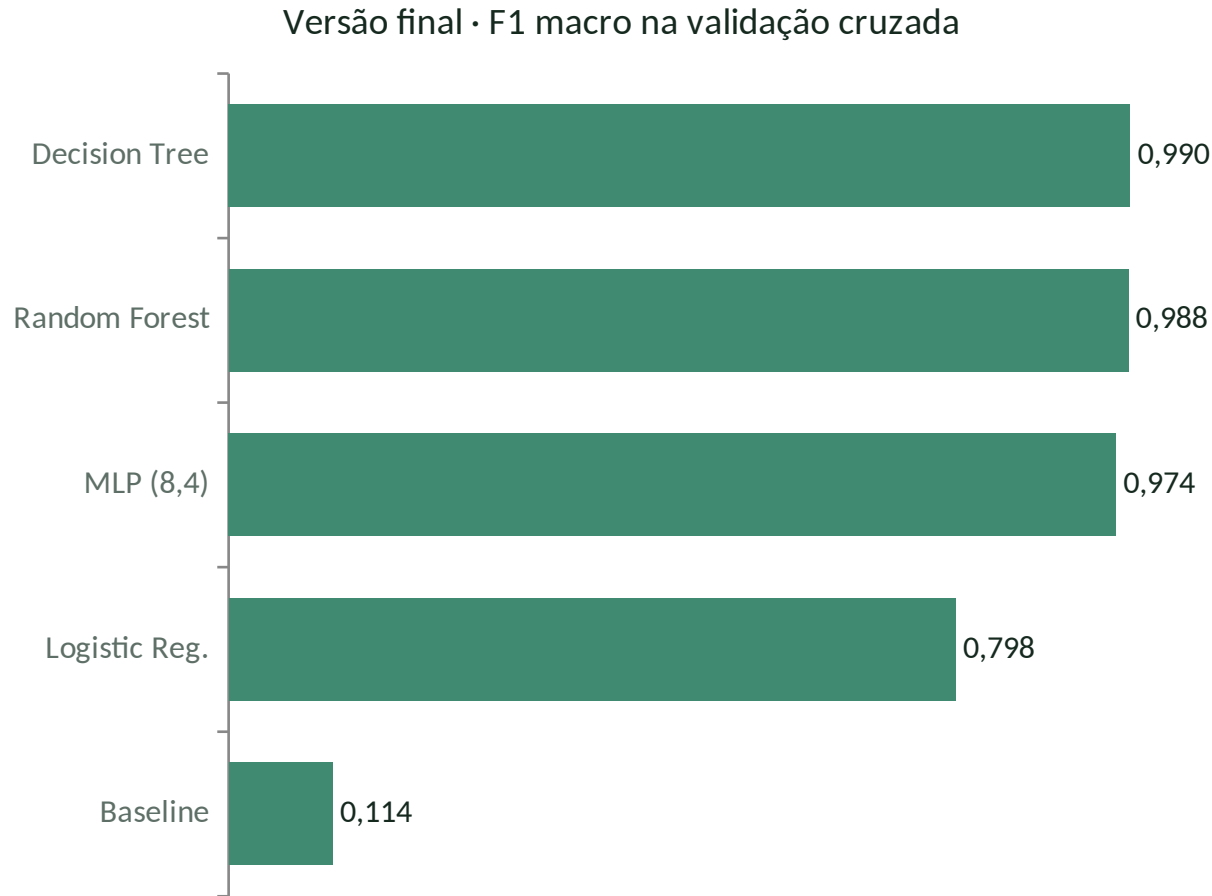
0,73 → 0,99

validação cruzada, antes e depois da expansão do dataset

O que mudou

Dados sintéticos concentrados perto dos limites, identificados pela coluna origem; split 70/15/15 e conjunto extra de generalização.

Cinco modelos comparados, um escolhido



MODELO SELECIONADO

Decision Tree

25 nós • profundidade 4 • ≤ 4 comparações



47× menos memória

Empata com a Random Forest no teste (0,993) com 400 B contra 18.944 B.



Interpretável

Cada decisão é uma sequência de comparações legíveis.



Sem normalização

Usa as unidades físicas direto, sem pré-processamento na placa.

Teste reservado com 127 combinações inéditas

99,2 %

accuracy

99,3 %

F1 macro

97,1 %

recall da classe crítica

98,2 %concordância com as regras em
20.000 entradas

Único erro: 1 caso crítico previsto como alerta, que
ainda aciona LED e buzzer.

Matriz de confusão

linhas: classe real · colunas: classe prevista

	Alerta	Crítico	Normal	Presença
Alerta	36	0	0	0
Crítico	1	33	0	0
Normal	0	0	20	0
Presença	0	0	0	37

Do scikit-learn ao firmware, sem bibliotecas de ML



Por que não quantização? Por que não só if/else?



Quantização (INT8)

- Serve para redes neurais: comprime pesos.
- A árvore não tem pesos, só 12 comparações.
- O modelo ocupa centenas de bytes em 264 kB.
- Arredondar 29,95 °C mudaria a decisão.
- A MLP exigiria TFLite Micro, mais pesado.

Sem pesos, não há o que quantizar.



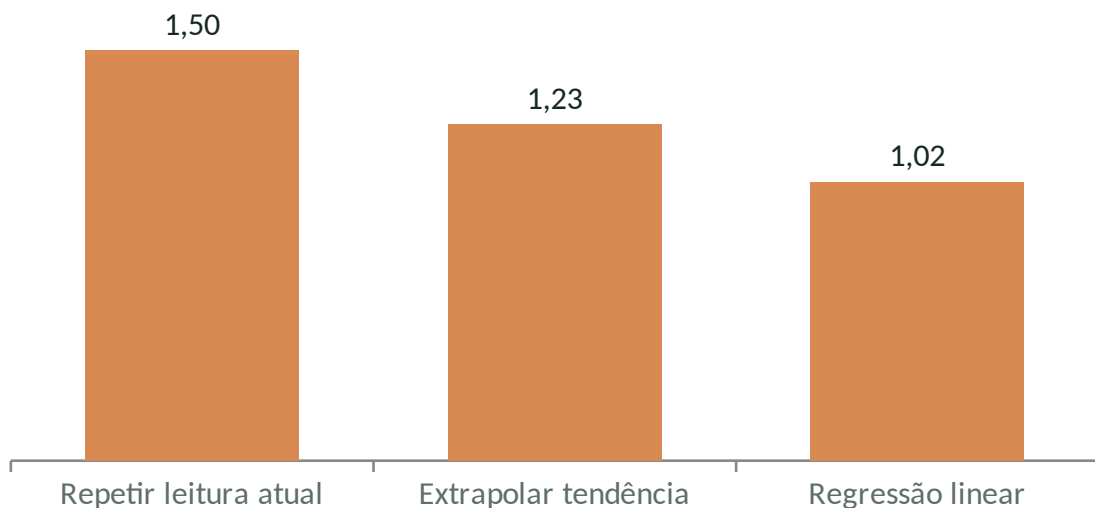
If/else escrito à mão

- O modelo já é if/else, gerado a partir dos dados.
- Os limiares foram aprendidos, não digitados.
- Regras manuais são o fallback fora do domínio.
- Com rótulos reais não há regra conhecida: basta retreinar.
- A previsão de +60 s é uma regressão, não um if.

O ML gera o if/else; as regras dão segurança.

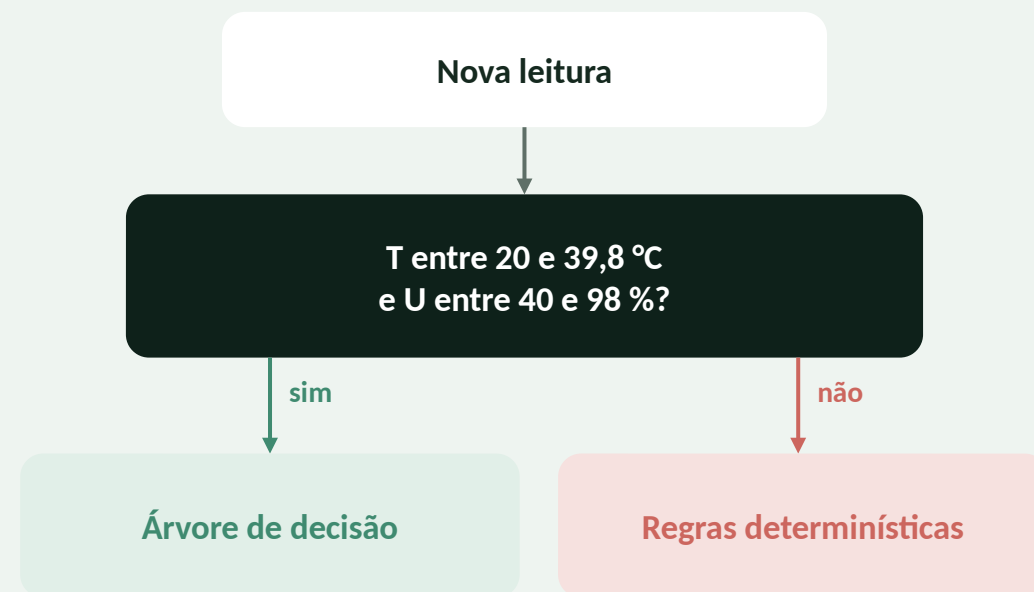
Antecipar o alerta e não confiar no modelo às cegas

Erro médio da previsão de +60 s (MAE, °C)



Regressão linear embarcada: 10 coeficientes sobre a janela das últimas 10 leituras. Se a temperatura prevista mudar a classe para pior, a placa sinaliza alerta_futuro.

Proteção fora do domínio de treino



Leitura inválida (NaN ou fora dos limites físicos): inferência suspensa e atuadores desligados.

Firmware gravado e rodando no Pico W

1

BOOTSEL

Segurar o botão ao conectar: a placa vira o drive RPI-RP2

2

UF2

Copiar tinymt_tree.uf2 para o drive

3

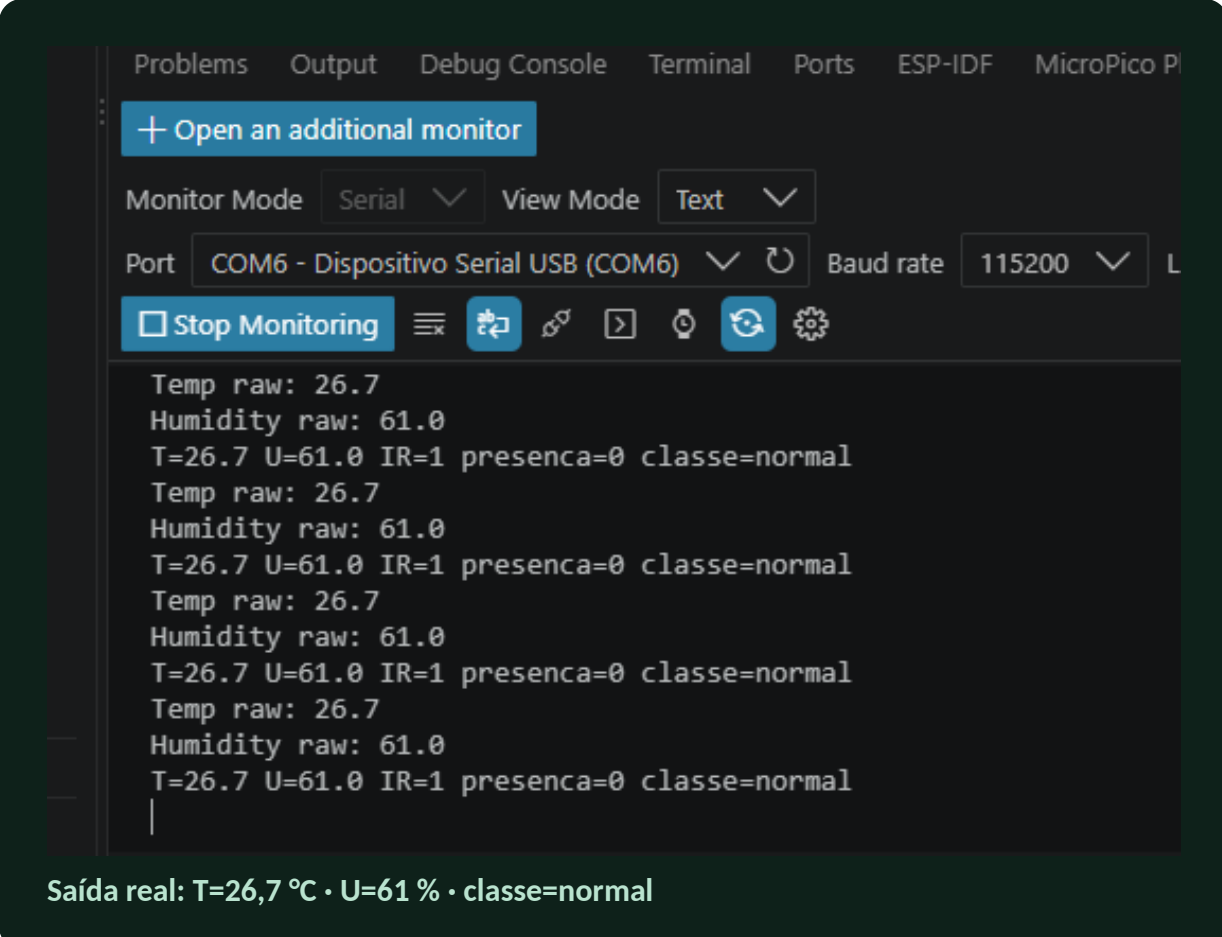
Reinício

O Pico inicia o firmware C++ automaticamente

4

Serial

Monitor a 115200 baud mostra a classe prevista



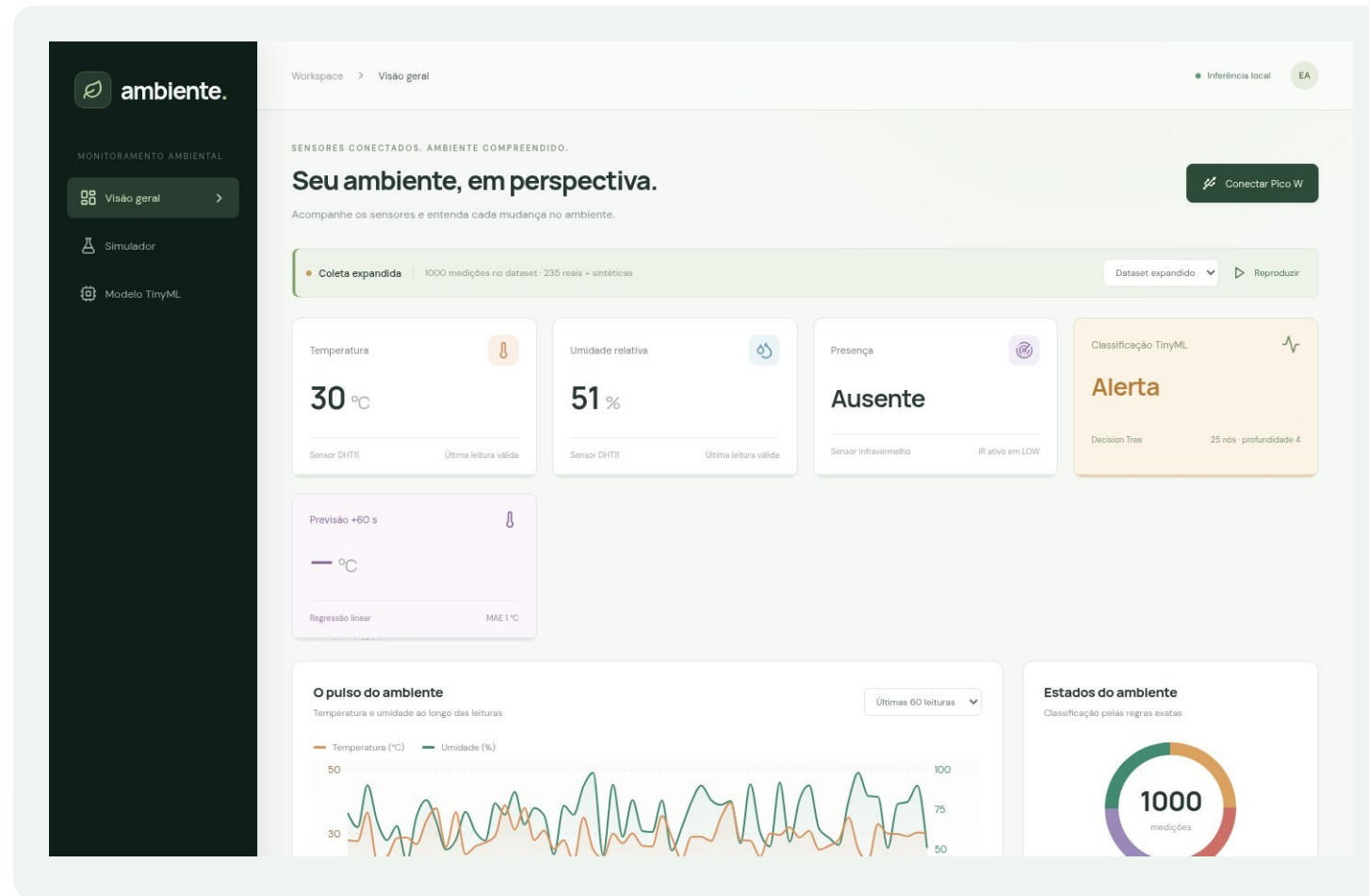
The screenshot shows the MicroPython IDE interface with the serial monitor open. The monitor is configured for 'Serial' mode, 'Text' view, port 'COM6 - Dispositivo Serial USB (COM6)', and a baud rate of '115200'. The output displays sensor data in a repeating pattern:

```
Temp raw: 26.7
Humidity raw: 61.0
T=26.7 U=61.0 IR=1 presenca=0 classe=normal
Temp raw: 26.7
Humidity raw: 61.0
T=26.7 U=61.0 IR=1 presenca=0 classe=normal
Temp raw: 26.7
Humidity raw: 61.0
T=26.7 U=61.0 IR=1 presenca=0 classe=normal
Temp raw: 26.7
Humidity raw: 61.0
T=26.7 U=61.0 IR=1 presenca=0 classe=normal
```

At the bottom of the IDE window, a summary of the real-time output is shown:

Saída real: T=26,7 °C • U=61 % • classe=normal

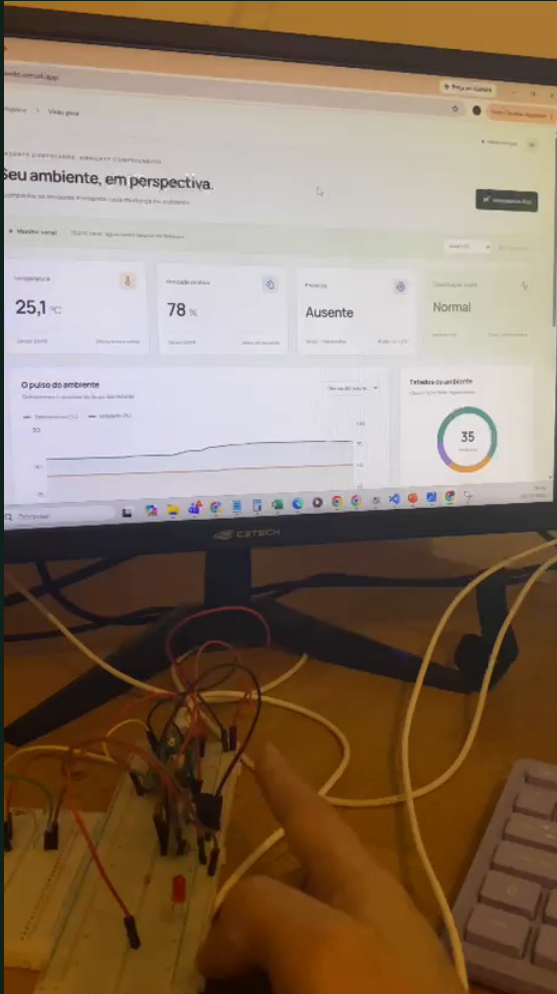
Do Pico W ao navegador, com segurança



-  **Pico W**
POST HTTPS a cada 10 s com token
-  **Edge Function**
Valida o token e o conteúdo da leitura
-  **PostgreSQL**
Restrições CHECK + Row Level Security
-  **Painel React**
Vercel · USB, nuvem e simulador

classificador-de-ambiente.vercel.app

Funcionando de ponta a ponta



A placa lê os sensores

DHT11 e IR na protoboard, ciclo de 2 s



Classifica sem computador

A árvore roda no RP2040 e aciona o LED



Envia por Wi-Fi

Leitura autenticada gravada no Supabase



O painel atualiza

O mesmo dado aparece no site em segundos

O notebook só exibe o site: todo o processamento acontece na placa.



Acesse o painel ao vivo

Resultados, limitações e próximos passos



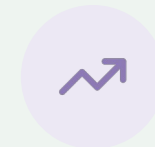
O que alcançamos

- Pipeline TinyML completo, do sensor ao firmware
- F1 macro de 99,3 % com 25 nós
- Previsão de +60 s e fallback seguro
- Nuvem segura e painel publicado



Limitações

- Rótulos derivados das regras
- 76 % do dataset é sintético
- Sem sessões ou horários na coleta real
- DHT11 com ± 2 °C e ± 5 % UR



Próximos passos

- Coletar sessões reais com rótulos observados
- Sensor mais preciso (SHT31) e histerese
- Medir latência e consumo na placa
- Guardar leituras offline e notificar alertas



Resultado: um pipeline TinyML completo, do sensor físico ao firmware executando no microcontrolador.

AGRADECEMOS A ATENÇÃO

Inteligência que cabe na placa

Perguntas?



Código e documentação

github.com/rafaelasantana-ia/classificador-de-ambiente

Painel ao vivo

classificador-de-ambiente.vercel.app

Rafaela Santana · Miriam Aguiar · Sara Coutinho
Prof. Rodrigo Kobashikawa Rosa · SENAI Santa Catarina

